Its syntax is as follows:

```
SELECT column1, column2...columnN
FROM table_name
[WHERE condition]

[ORDER BY column1, column2, .. columnN] [ASC | DESC];
```

For instance, the statement below sorts a column in the ascending order by age:

```
testdb=# SELECT * FROM student_table ORDER BY age ASC;
```

We can use the *DISTINCT* keyword to eliminate duplicate records. For instance, the following statement lists only unique values:

```
testdb=# SELECT DISTINCT * FROM student_table;
```

Advanced SQL

In this section, we'll discuss some of the advanced features of PostgreSQL like built-in functions, transactions, the import/export feature and programming language interfaces.

Built-in functions

PostgreSQL has many useful built-in functions and this section discusses some of the more important among them.

1. *COUNT*: As the name suggests, this is used to count the number of records in a table. The statement below shows the usage:

```
testdb=# SELECT COUNT(*) FROM student_table;
```

2. *MAX:* This is used to find out the records with the maximum value. For instance, the statement below finds the record with the maximum age:

```
testdb=# SELECT MAX(age) FROM student_table;
```

3. *MIN:* This is used to find out the records with the minimum value. For instance, the statement below finds the record with the minimum age:

```
testdb=# SELECT MIN(age) FROM student_table;
```

4. *AVG:* This is used to calculate the average of a record set. For instance, the statement below calculates the average of the ages:

```
testdb=# SELECT AVG(age) FROM student_table;
```

5. *SUM:* This function is used to calculate the sum of the record set. For instance, the statement below calculates the sum of the ages:

```
testdb=# SELECT SUM(age) FROM student_table;
```

6. *ARRAY_AGG:* The *array aggregate* function is used to concatenate the input values into an array as shown below:

```
testdb=# SELECT ARRAY_AGG(age) FROM student_table;
```

This statement will show the aggregate values as follows:

```
{21,22,22,21}
```

In addition to this, PostgreSQL supports various numeric and string related functions. You can refer to the official documentation to know more details.

Transactions

Transactions are a fundamental concept of all database systems. The essential point of a transaction is that it bundles multiple steps into a single, all-or-nothing operation. Transactions ensure data integrity.

PostgreSQL supports the following transaction control commands:

1) *BEGIN:* Starts a new transaction
2) *COMMIT:* Saves changes to databases
3) *ROLLBACK:* Rolls back the changes

Note that transactional control commands are only used with the DML statements like *INSERT*, *UPDATE* and *DELETE*. They cannot be used while creating tables or dropping them, because these operations are automatically committed in the database.

Let us look at examples of rollback first. In this example, we'll perform the following actions:

1) Begin the transaction
2) Delete a record from the database
3) Using the *SELECT* query, ensure that the record is deleted
4) Roll back a transaction
5) Using the *SELECT* query, ensure that the record is added back

```
testdb=# BEGIN;                                    # Step-1

testdb=# DELETE FROM student_table WHERE id = 1; # Step-2

testdb=# SELECT * FROM student_table;              # Step-3

testdb=# ROLLBACK;                                 # Step-4

testdb=# SELECT * FROM student_table;              # Step-5
```

Let us look at an example of commit. In this example, we'll perform the following actions:

1) First, begin the transaction